

AEGIS client upload options

AEGIS supports **six** distinct ways for a study to enter the system. Each has a different privacy posture, install footprint, throughput profile, and target user. This doc explains them all and helps you pick the right one for a given situation.

At a glance

#	Entry point	Where PHI is de-identified	Install	Throughput	Best for
1	Light web client	Cloud	Zero	One study at a time	Casual contributors
2	Heavy web client	User's browser	Zero (~10 MB lazy on first use)	One study at a time, slow	Privacy-sensitive contributors
3	Desktop app (<i>Tauri</i>)	User's machine (native)	One installer per OS	Many studies, native speed	Power users (research coordinators)
4	AEGIS Router	On-prem site	docker compose up at the site	Full PACS auto-route, high volume	High-throughput hospitals/labs
5	Cloud-side DIMSE pull	Cloud	Firewall rule only	Pull from remote PACS via C-MOVE	Sites that can't run a router but can open a port
6	Direct API upload	Wherever the script runs	API key + curl/SDK	Scriptable, programmatic	Pipelines, CI, MCP agent

The first three are user-facing UIs. The router is a sysadmin install. Cloud DIMSE pull and the direct API are alternate machine-to-machine entry points that aren't really "client tiers" so much as additional doors into the same backend.

1. Light web client

The default upload portal at <https://upload.aegisimaging.ai>. Already in production today.

What runs in the browser:

- DICOM parsing (dcmjs)
- PS3.15 Annex E Basic Profile tag de-identification
- Date shifting
- Pseudonymization of PatientID + PatientName (deterministic SHA-256 with a per-project salt)

What the cloud does:

- Pixel-level PHI scrubbing (OCR via Tesseract / Cloud Vision / Gemini / etc.)
- Face de-identification (mri_deface, DeepDefacer, mri_reface)

- QC checks, classification, protocol compliance
- BIDS conversion, analytics
- Routing rules to forwarding destinations

When to use it:

- Casual contribution of one or a handful of studies
- Users on managed devices who shouldn't install anything
- Quick "drag and drop" workflow
- When the receiving project trusts the cloud's processing

Bundle size: ~1 MB JS.

Bulk: Drag a parent folder with multiple studies → the portal detects each `StudyInstanceUID` and uploads them sequentially with per-study progress.

2. Heavy web client

The same upload portal as #1, with an opt-in toggle for **on-device deep de-identification**. Enabled per-upload from the preview screen.

What runs in the browser (with heavy mode enabled):

- Everything from #1, plus:
- **Pixel PHI scrub** — Tesseract.js (WASM, lazy-loaded ~10 MB) decodes uncompressed and JPEG-baseline DICOM, runs OCR per frame, redacts PHI-shaped tokens (dates, names, MRNs, accession numbers, etc.) with black boxes, re-encodes into the dataset's PixelData.
- **Face de-id** — for head MR/CT studies. Composes a 3D voxel volume from the slices, applies an anterior-zero heuristic (or a TF.js model when one is configured), writes the defaced bytes back into the dataset.

What the cloud does:

- Nothing for PHI removal — it's all done before upload.
- Still runs QC, classification, BIDS, analytics, routing — these don't reveal PHI.

When to use it:

- IRB protocols mandating on-device de-id
- Regulatory environments that can't send pixel PHI to a third party
- Researchers contributing handfuls of studies who want the strongest privacy posture
- "Defense in depth" — heavy mode + server-side scrub catches anything the browser missed

Bundle size:

- Default chunk unchanged from light mode (peer-dep import)
- First heavy-mode toggle: adds ~10 MB cached download for Tesseract.js + WASM core + language data
- Face de-id model (when configured): adds ~30 MB cached download for the TF.js weights

Performance:

Image type	Pixel scrub per frame	Face de-id per study
256×256 MR slice	0.3–0.8 s	n/a (volume-level)

512×512 CT slice	0.5–1.2 s	n/a
1024×1024 secondary capture	1–2 s	n/a
200-slice head T1	60–180 s total	5–10 s (anterior heuristic), 30–120 s (TF.js model)

Bulk: Same as light mode — drag many studies, each gets the full heavy-mode pipeline. Tesseract worker is reused across files in a session.

3. Desktop app (*coming next*)

A Tauri-based native installer for Mac, Windows, and Linux that wraps the same React upload portal as #1/#2. Cross-platform from one codebase.

Why it exists when the web client already does everything:

- **Native filesystem access** — drag a folder from Finder/Explorer, watch a "drop folder" for new studies and auto-upload them.
- **No 10 MB browser download** — the WASM + Tesseract assets are bundled inside the installer, so first-use latency disappears.
- **Code-signed binaries** — Mac notarization + Windows Authenticode, no scary "unidentified developer" warning.
- **Native ML acceleration** — wraps native Tesseract / ONNX Runtime via Tauri commands for ~10× speedup on face de-id versus WASM in the browser.
- **Offline mode** — if the cloud is unreachable, studies queue locally and ship when the link comes back.

What runs natively:

- Everything from #2 (heavy mode), but accelerated.
- Optional: watch-folder daemon, native menu, auto-update.

Installer size: ~15-30 MB depending on whether the face de-id model is bundled.

When to use it:

- Research coordinators uploading studies routinely (daily / weekly)
- Clinical assistants who want a drop folder
- Demoing on a laptop without spinning up Docker

Status: PR pending. The architectural foundation is in place — the desktop app reuses 100% of the `@aegis/client` library and the upload-portal React UI from #1 and #2.

4. AEGIS Router (on-prem)

For sites that want **every study they read** to flow into AEGIS without human intervention.

Documented in detail in [router/README.md](#) and [router/docs/spoke-deployment.md](#).

Two install scenarios:

- **Single host** (laptop, NUC, lab workstation): `docker compose up -d` brings up the router. Profile-gated sidecars (`--profile defacing` , `--profile phi` , etc.) opt in to additional

capabilities.

- **Production spoke:** `aegis-router-init --site-token=<TOKEN>` provisioned by the cloud admin, then the same `docker compose up`. Onboarding takes ~5 minutes.

Capabilities:

- DICOM C-STORE SCP on port 11112 (the PACS pushes to it)
- HTTPS upload endpoints on port 8080 (same contract as the cloud upload-portal)
- mTLS forwarding to the cloud receiver
- Local quarantine (SQLite) for studies that fail de-id
- Prometheus metrics for throughput benchmarking
- Operator API for retry / clear / inspect

When to use it:

- Hospitals with a PACS that should auto-route everything to a project
- High-throughput sites (>100 studies/day) where browser uploads aren't scalable
- IRB protocols that mandate on-prem de-id
- Air-gapped sites (the router can run fully locally with cloud forwarding off)

Performance: Limited by PACS push throughput. Tested at 500+ studies/day on a 2-vCPU VM.

5. Cloud-side DIMSE pull

For sites that can't or won't install the router but **can** open a firewall port. The cloud's `dimse-receiver` service supports DICOM C-MOVE — an admin configures the remote PACS as a known DIMSE peer, and the cloud reaches out to pull specific studies on demand.

Setup:

1. Site IT opens an inbound rule on their PACS for the cloud's egress IP range on TCP/11112.
2. Cloud admin adds the peer to the AEGIS DIMSE peer table: `POST /api/dimse-peers { ae_title, host, port, ... } .`
3. Admin or scripted client triggers a pull: `POST /api/dimse/retrieve { peer_id, study_instance_uid, ... } .`

When to use it:

- Sites that say "we can't run software, but we can open a firewall hole"
- Pulling specific studies on demand rather than every study
- Filling in historical data that's still on the PACS

Privacy posture caveat: This is the weakest of the six. The DICOM bytes traverse the link from the PACS to the cloud unprocessed; de-id only happens *after* arrival. If on-prem de-id is required, use the router (#4).

6. Direct API upload

The same `/api/upload/init` → `PUT /api/upload/file/...` → `/api/upload/complete` flow that the upload portal uses, but driven from a script with an API key.

Typical use:

```
TOKEN=$(curl -s https://aegisimaging.ai/api/auth/api-key/... | jq -r .key)
# Use @aegis/client's uploadStudy() from Node, or shell out the three calls
```

Authentication:

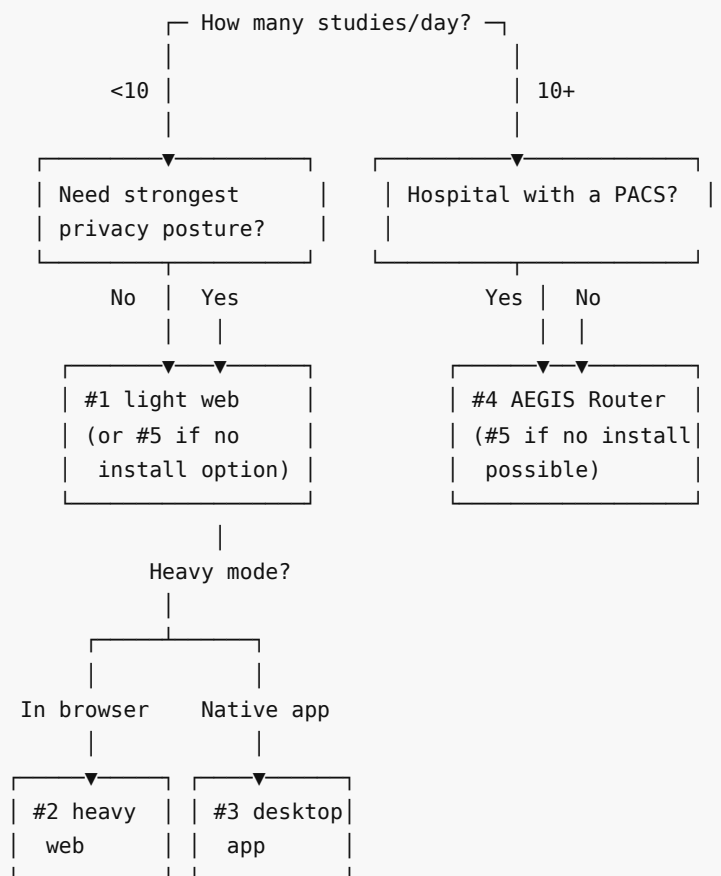
- API key in `Authorization: Bearer <key>` header
- Keys are minted from the admin dashboard (Settings → API Keys)
- Scoped to a project, optionally expiring

When to use it:

- CI/CD that uploads QA studies (e.g. synthetic data for regression tests)
- Migration scripts pulling from an existing archive
- MCP server / agent-driven workflows
- Headless lab pipelines

De-id responsibility: Whichever side runs the script is responsible. For unprocessed PHI-bearing studies, run the de-id pipeline locally first (the `@aegis/client` library exposes `deidentify()`, `scrubInstance()`, and `defaceStudy()` for this).

Picking the right tier — flowchart



Doesn't fit any of the above?

└─ Scripted/automated → #6 direct API

Bulk upload (cross-cutting)

All six entry points support bulk upload via the same `@aegis/client.bulkUpload()` primitive:

Tier	How bulk works
#1 Light web	Drag parent folder → multi-study table → sequential upload (configurable concurrency up to 4)
#2 Heavy web	Same as #1 + heavy mode runs per study
#3 Desktop	Same as #2 + watch-folder support so dropping new folders triggers upload automatically
#4 Router	PACS pushes whatever it pushes — bulk is implicit
#5 DIMSE pull	One C-MOVE per study, scripted
#6 API	Loop over studies in your script; per-study upload reuses the same flow

The library:

```
import { bulkUpload } from '@aegis/client'

await bulkUpload(files, {
  projectSlug: 'my-project',
  concurrency: 3,
  pixelScrub: { enabled: true },
  faceDeid: { enabled: true },
  onProgress: p => console.log(`${p.studiesCompleted} / ${p.totalStudies}`),
  onStudyDone: s => console.log(`done: ${s.studyInstanceId} → ${s.status}`),
})
```

Side-by-side: privacy posture vs throughput

high		★ #4 Router (high-throughput, on-prem)
thru-		
put		★ #3 Desktop (native bulk + watch folder)
		★ #2 Heavy ★ #5 Cloud DIMSE pull
		★ #1 Light
low		★ #6 API (depends on script throughput)

low	on-device PHI protection	high
-----	--------------------------	------

★ Lower-left = casual/quick. Upper-right = serious deployment.

Configuration matrix

Setting	Light (#1)	Heavy (#2)	Desktop (#3)	Router (#4)
salt (pseudonym hash)	Project-scoped	Same	Same	MIDI_B_SALT env var per site
retainedTags (keep through de-id)	Anon profile	Same	Same	MIDI_B_RETAINED_TAGS env var
dateShift	Optional per-upload	Same	Same	Always-on per site
pixelScrub	Server-side	In-browser	Native	Server-side via phi-detection sidecar
faceDeid	Server-side	In-browser	Native	Server-side via defacing sidecar
mTLS auth	n/a (browser session)	n/a	n/a	Per-spoke cert from cloud
API key auth	n/a	n/a	n/a	n/a (uses mTLS)
Quarantine on failure	Server-side review	Same	Same	Local SQLite at the site

Built with [Claude Code](#). For questions, see the [demo setup guide](#) or open an issue on GitHub.